CourseHub

CollegeCourseRegistrationSystem

Real-WorldExamplesandUse Cases
PostgreSQL + Neo4j + Next.js DBMS mini project

1. Introduction

CourseHub isa full-stack DBMS project that manages students, courses, faculty, enrollments, attendance and marks. It uses **PostgreSQL** for structured academic records and **Neo4j** for course prerequisite relationships, with **role-based login** (Admin and Student) built on JWT and bcryptjs. This document shows how each part of the project maps to situations that happen in real colleges and universities, so the design choices can be explained with concrete examples.

2. Where This Kind of System Is Used

Real-world setting	How CourseHub's design applies
University registrar's office	Admin staff maintain student, course and faculty records and manage enrollments from one dashboard.
Engineering / arts & science colleges	Each semester, students are enrolled in courses; attendance and marks are tracked against those enrollments.
Online learning platforms	Learning paths with prerequisites (for example, a beginner course before an advanced one) are naturally modelled as a graph.
Corporate training programs	Employees enroll in training modules, attendance is recorded, and assessments are graded.
Student self-service portals	Students log in and view only their own profile, enrollments, attendance and marks.

3. Real-World Scenarios

Scenario A: Start of semester - enrolling a student

A registrar admits a new student and must put them in the right courses. In CourseHub the admin creates the student record, then creates enrollment records linking that student to courses. Because enrollments references both students and courses through foreign keys, the database refuses an enrollment for a student or course that does not exist.

Concepts shown: primary and foreign keys, relational constraints, CRUD operations, SQL joins.

Scenario B: Checking course prerequisites

In a real curriculum, a student typically cannot take Database Management Systems before completing earlier programming and data structure courses. The repo's Neo4j setup creates five courses (C101 Programming Fundamentals, C102 Data Structures, C103 Database Management Systems, C104 Operating Systems, C105 Computer Networks) and five prerequisite relationships between them.

The prerequisite structure is stored as `(Course)-[:HAS_PREREQUISITE]->(PrerequisiteCourse)`. Graph databases suit this well because chains of dependencies (A needs B, B needs C) are simple to follow, while in SQL they require recursive joins. An admin can view these relationships on the Prerequisites page.

```
MATCH (c:Course)-[:HAS_PREREQUISITE]->(p:Course)
RETURN c.course_id, c.course_name,
       p.course_id, p.course_name;
```

Concepts shown: graph modelling, nodes and relationships, Cypher queries, polyglot persistence (using two databases for what each does best).

Scenario C: Mid-semester attendance tracking

Colleges commonly require a minimum attendance percentage for exam eligibility. Admins record attendance per student and course, and each student can open their Student Dashboard to see their own attendance records instead of asking the office.

Concepts shown: one-to-many relationships, views, indexes for fast lookups, per-user data access.

Scenario D: End of semester - marks and grades

After exams, staff enter marks and grades for each enrollment. Students then log in and see only their own results. Admins can review marks for all students through the Marks page.

Concepts shown: SQL functions and triggers (the schema includes these), data integrity, role-based access.

Scenario E: Assigning faculty to courses

Departments assign instructors to the courses they teach, and one faculty member may teach several courses while a course may have several instructors. The faculty_courses table resolves this many-to-many relationship.

Concepts shown: many-to-many relationships, junction tables, normalization.

Scenario F: Protecting sensitive student data

Student records are private. CourseHub separates access by role: admin pages and admin-only API endpoints are protected on the server, and a student is restricted to their own dashboard and cannot open admin pages or protected endpoints. Passwords are hashed with bcryptjs, sessions use JWT, and database credentials and the session secret live in environment variables rather than in the code.

Concepts shown: authentication vs. authorization, password hashing, session management, secrets handling.

4. Mapping Real-World Needs to Database Tables

Real-world need	Table / store	Technology
Student identity and profile	students	PostgreSQL
Course catalogue	courses	PostgreSQL
Teaching staff	faculty	PostgreSQL
Who teaches what	faculty_courses	PostgreSQL
Who is taking what	enrollments	PostgreSQL
Class attendance	attendance	PostgreSQL
Exam results	marks	PostgreSQL
Login accounts and roles	app_users	PostgreSQL
Prerequisites in SQL	course_prerequisites	PostgreSQL
Prerequisite dependency graph	Course nodes + HAS_PREREQUISITE	Neo4j

5. A Day in the Life

Who	What they do in CourseHub
Registrar (Admin)	Logs in, adds a new student, enrolls them in courses, and checks the prerequisite graph for a course.
Faculty-side admin	Records attendance after a class and enters marks after an exam.
Student	Logs in, sees enrolled courses and status, then checks attendance and grades. Cannot see other students' data or admin pages.